

- Document 09: FHIR Integration & Payer Policy Gate
 - CodePerfect Auditor — HL7 FHIR R4 Claim Builder & Intelligent Auto-Adjudication
 - Overview
 - Part 1: FHIR R4 Claim Builder — `services/fhir_claim_builder.py`
 - What is FHIR?
 - ICD Coding System URI Mapping
 - Explanation
 - Patient Resource — Embedded contained Sub-Resource
 - Explanation
 - Diagnosis Entries — Multi-Code FHIR Structure
 - Explanation
 - CPT Line Item Entries — Procedure Coding
 - Explanation
 - The FHIR Claim Resource — Final Assembly
 - Explanation — Custom Extensions
 - Part 2: Payer Policy Gate — `services/payer_policy_gate.py`
 - Design Philosophy
 - Trust Gate Classification
 - Explanation
 - The Gate Runner — All 7 Checks
 - Check 1: Required Demographics
 - Check 2: ICD Version Compatibility
 - Check 3: Mapping Quality Gate
 - Check 4 & 5: AI Score Thresholds with Coding Mode Adjustment
 - Check 6: Procedure Code Presence
 - Check 7: Discrepancy Type Filter
 - Check 8: Custom Payer Rules (`_evaluate_custom_rules`)
 - Gate Final Decision
 - Real Gate Report Examples
 - Integration Flow — How FHIR and Gate Work Together

Document 09: FHIR Integration & Payer Policy Gate

CodePerfect Auditor — HL7 FHIR R4 Claim Builder & Intelligent Auto-Adjudication

Project: CodePerfect Auditor | **Hackathon:** Jatayu Hackathon **Team:** AgentsCrew — Nanda Kishore R, Subashini S, Nathin R **Institution:** Saveetha Engineering College | **Date:** 31-03-2026

Overview

This document covers two advanced services that bridge the gap between the AI coding pipeline and the real-world insurance billing ecosystem:

1. **FHIR R4 Claim Builder** ([services/fhir_claim_builder.py](#)) — Transforms the AI pipeline output into a standards-compliant HL7 FHIR R4 **Claim** resource that any FHIR-compatible payer or clearinghouse can ingest natively.
2. **Payer Policy Gate** ([services/payer_policy_gate.py](#)) — A deterministic rule engine that evaluates every submitted claim against the payer's configured policy thresholds to decide: auto-approve, or flag for manual adjudicator review.

These two services are the commercial core of CodePerfect — they are what transform a coding tool into a full **Revenue Cycle Management (RCM) automation platform**.

Part 1: FHIR R4 Claim Builder — [services/fhir_claim_builder.py](#)

What is FHIR?

HL7 FHIR (Fast Healthcare Interoperability Resources) is the international standard for exchanging electronic health records and clinical billing data. Mandated by the Centers

for Medicare and Medicaid Services (CMS) as the standard for all US payer-to-provider data exchange since 2021.

CodePerfect builds a FHIR R4 **Claim** resource — the structured JSON artifact that travels from the hospital to the insurance payer to formally request reimbursement.

ICD Coding System URI Mapping

```
# FHIR requires a coding system URI to identify WHICH ICD classification a code
belongs to.
# The same code (e.g. "E11.22") exists in both ICD-10-CM and ICD-11 – the URI
disambiguates.
_ICD_SYSTEM_MAP = {
    "ICD-11": "http://id.who.int/icd/release/11/mms",
    "ICD-10": "http://hl7.org/fhir/sid/icd-10-cm",
    "ICD-10-CM": "http://hl7.org/fhir/sid/icd-10-cm",

    # Pipeline mapping path → URI (fallback when icd_version not set)
    "who_api_icd11": "http://id.who.int/icd/release/11/mms", # WHO API hit
for ICD-11
    "who_api_icd10": "http://hl7.org/fhir/sid/icd-10-cm", # WHO API hit
for ICD-10
    "local_embedding": "http://hl7.org/fhir/sid/icd-10-cm", # Local DB
vector search
    "provider_fallback": "http://hl7.org/fhir/sid/icd-10-cm", # FHIR Condition
fallback
    "provider_augmented": "http://hl7.org/fhir/sid/icd-10-cm",
}

_CPT_SYSTEM = "http://www.ama-assn.org/go/cpt" # Official AMA CPT coding system
URI

def _icd_system_for(icd_version: Optional[str], mapping_path: Optional[str]) ->
str:
    """
    Priority: mapping_path (most specific) → icd_version → default ICD-10-CM.
    The mapping_path carries first-class evidence of which classification system
    was used during the pipeline run. Use that before falling back to org settings.
    """
    if mapping_path and mapping_path in _ICD_SYSTEM_MAP:
        return _ICD_SYSTEM_MAP[mapping_path]
    if icd_version and icd_version in _ICD_SYSTEM_MAP:
        return _ICD_SYSTEM_MAP[icd_version]
    return _ICD_SYSTEM_MAP["ICD-10-CM"]
```

Explanation

The ICD system URI is one of the most important fields in a FHIR Claim. Without it, a receiving payer's FHIR server cannot validate the diagnosis codes and may reject the entire claim with a validation error. The priority order ensures that if the pipeline used the WHO ICD-11 API for resolution (indicated by `mapping_path = "who_api_icd11"`), the FHIR resource correctly declares the WHO MMS URI — even if the org's default in `org_settings` is still set to ICD-10.

Patient Resource — Embedded contained Sub-Resource

```
def build_fhir_claim_proposal(
    *,
    claim_id: str,
    session_id: str,
    organization_id: str,
    organization_name: str,
    payer_id: str,
    payer_name: str,
    patient_name: Optional[str],
    patient_dob: Optional[str],          # YYYY-MM-DD
    patient_sex: Optional[str],          # M | F | other | unknown
    icd_codes: list,
    cpt_codes: list,
    financial_summary: dict,
    icd_version: Optional[str],
    mapping_path: Optional[str],
    total_billed_amount: float,
) -> dict:

    # — Patient resource (inline contained) —————
    # FHIR allows embedding related resources directly inside a Claim using the
    # 'contained' array. This makes the Claim self-contained – no external
    # Patient resource lookup required by the receiving payer.
    patient_resource: dict = {
        "resourceType": "Patient",
        "id": "patient-1",          # Local reference ID used in Claim.patient.reference
    }
    if patient_name:
        parts = patient_name.strip().split(" ", 1)
        patient_resource["name"] = [
            {
                "use": "official",
                "text": patient_name,          # Full name
                "family": parts[-1] if len(parts) > 1 else patient_name, # Last
                "given": [parts[0]] if len(parts) > 1 else [],          # First name
            }
        ]
    ]
```

```

if patient_dob:
    patient_resource["birthDate"] = patient_dob # FHIR uses YYYY-MM-DD format

if patient_sex:
    sex_map = {"M": "male", "F": "female", "OTHER": "other", "UNKNOWN":
"unknown"}
    # FHIR uses lowercase gender codes – CMS uses M/F – we normalize here
    patient_resource["gender"] = sex_map.get(patient_sex.upper(), "unknown")

```

Explanation

The FHIR `Patient.gender` field uses lowercase values ("male", "female") while payer systems typically store M/F. The `sex_map` performs this normalization so the produced FHIR resource passes W3C FHIR validator checks without errors. If `patient_name` or `patient_dob` was not extracted from the clinical document (common with hand-written charts), those fields are simply omitted rather than populated with placeholder data — FHIR validators reject placeholder values like "Unknown Patient" in structured name fields.

Diagnosis Entries — Multi-Code FHIR Structure

```

diagnosis_entries = []
for idx, icd in enumerate(icd_codes or []):
    code = icd.get("code") or icd.get("ai_icd_code") or ""
    desc = icd.get("description") or ""

    entry: dict = {
        "sequence": idx + 1, # FHIR requires 1-based sequential numbering
        "diagnosisCodeableConcept": {
            "coding": [
                {
                    "system": icd_system, # The URI from _icd_system_for()
                    "code": code, # e.g. "E11.22"
                    "display": desc, # Human-readable description
                }
            ],
            "text": desc,
        },
    }
    # The FIRST code in the list is marked as "principal" (the primary billing
diagnosis).
    # Subsequent codes are additional diagnoses (comorbidities).
    if idx == 0:

```

```

        entry["type"] = [
            {
                "coding": [
                    {
                        "system": "http://terminology.hl7.org/CodeSystem/ex-
diagnostictype",
                        "code": "principal",
                    }
                ]
            }
        ]
        diagnosis_entries.append(entry)

```

Explanation

The **sequence** field links diagnosis entries to claim items (CPT codes). An item's **diagnosisSequence: [1]** means "this procedure was clinically justified by the first diagnosis in this list." This is the FHIR equivalent of the EDI SV107 diagnosis code pointer. Payer adjudication engines use this link to verify that the billed procedure is clinically appropriate for the stated diagnosis — a critical check for fraud detection.

CPT Line Item Entries — Procedure Coding

```

for idx, item in enumerate(line_items):
    cpt_code = item.get("cpt_code") or item.get("code") or ""
    base_price = float(
        item.get("gross_charge") or
        item.get("base_price") or
        item.get("cms_base_price") or 0.0
    )

    item_entry: dict = {
        "sequence": idx + 1,
        "productOrService": {
            "coding": [
                {
                    "system": _CPT_SYSTEM, # "http://www.ama-
assn.org/go/cpt"
                    "code": cpt_code, # e.g. "93306"
                    "display": cpt_meta.get("description") or "",
                }
            ]
        },
        "unitPrice": {
            "value": round(base_price, 2),
            "currency": "INR", # India-first: Indian Rupees

```

```

    },
    "net": {
        "value": round(base_price * float(item.get("quantity") or 1),
2),
        "currency": "INR",
    },
}
# Link each procedure to the primary diagnosis sequence (ICD code [1])
if diagnosis_entries:
    item_entry["diagnosisSequence"] = [1]
item_entries.append(item_entry)

```

Explanation

The `currency: "INR"` reflects the India-first deployment context of CodePerfect Auditor targeting domestic insurance payers (like Star Health, HDFC ERGO, National Insurance) and government schemes (Ayushman Bharat, CGHS). This is a deliberate localization decision distinguishing the platform from US-centric RCM tools.

The FHIR Claim Resource — Final Assembly

```

fhir_claim: dict = {
    "resourceType": "Claim",
    "id": claim_id,

    # meta.tag: machine-readable provenance marker
    # "hospital-proposed" indicates this artifact has NOT been payer-verified.
    # After payer adjudication, the tag should change to "payer-verified".
    "meta": {
        "profile": ["http://hl7.org/fhir/StructureDefinition/Claim"],
        "lastUpdated": now_iso,
        "tag": [
            {
                "system": "http://integronix.io/tags",
                "code": "hospital-proposed",
                "display": "Hospital Proposed – not yet payer-verified",
            }
        ],
    },

    "status": "active", # FHIR Claim status lifecycle: active → cancelled →
enteredinerror
    "type": {
        "coding": [{
            "system": "http://terminology.hl7.org/CodeSystem/claim-type",
            "code": "institutional", # 'professional' | 'institutional' |
'oral' | 'vision'

```

```

        "display": "Institutional",
    }]
},
"use": "claim",          # 'claim' | 'preauthorization' | 'predetermination'

# Patient reference uses local '#patient-1' anchor pointing to contained
resource
"patient": {
    "reference": "#patient-1",
    **({"display": patient_name} if patient_name else {})
},

"created": now_iso,
"insurer": {"display": payer_name, "identifier": {"value": payer_id}},
"provider": {"display": organization_name, "identifier": {"value":
organization_id}},

"diagnosis": diagnosis_entries,
"item":      item_entries,

"total": {"value": round(total_billed_amount, 2), "currency": "INR"},

# Self-contained: payer doesn't need to query a separate Patient resource
"contained": [patient_resource],

# Custom extensions: non-breaking, carry pipeline provenance metadata
"extension": [
    {
        "url":
"http://integronix.io/fhir/StructureDefinition/session-id",
        "valueString": session_id,          # Links FHIR Claim back to our
coding session
    },
    {
        "url":      "http://integronix.io/fhir/StructureDefinition/icd-
version",
        "valueString": icd_version or "ICD-10",
    },
    {
        "url":
"http://integronix.io/fhir/StructureDefinition/mapping-path",
        "valueString": mapping_path or "unknown", # Audit trail: how codes
were selected
    },
    {
        "url":
"http://integronix.io/fhir/StructureDefinition/proposal-status",
        "valueString": "HOSPITAL_PROPOSED",
    },
],
}
return fhir_claim

```

Explanation — Custom Extensions

FHIR **extension** elements allow custom data to be attached to any FHIR resource without violating the standard — any FHIR server that doesn't understand them simply ignores them. Our extensions carry the **session_id** (for tracing back to the AI pipeline run), **icd-version** (for payer compatibility checking), and **mapping-path** (for the Payer Policy Gate to evaluate code quality).

A real example of the produced FHIR JSON:

```
{
  "resourceType": "Claim",
  "id": "de305d54-75b4-431b-adb2-eb6b9e546014",
  "status": "active",
  "type": {"coding": [{"code": "institutional", "display": "Institutional"}]},
  "use": "claim",
  "patient": {"reference": "#patient-1", "display": "John Smith"},
  "created": "2026-03-31T01:15:00Z",
  "insurer": {"display": "Star Health Insurance", "identifier": {"value": "payer-uuid"}},
  "provider": {"display": "City General Hospital", "identifier": {"value": "org-uuid"}},
  "diagnosis": [
    {
      "sequence": 1,
      "type": [{"coding": [{"code": "principal"}]}],
      "diagnosisCodeableConcept": {
        "coding": [{"system": "http://hl7.org/fhir/sid/icd-10-cm", "code": "E11.22", "display": "Type 2 DM with CKD"}],
        "text": "Type 2 DM with CKD"
      }
    }
  ],
  "item": [
    {
      "sequence": 1,
      "productOrService": {"coding": [{"system": "http://www.ama-assn.org/go/cpt", "code": "99233"}]},
      "unitPrice": {"value": 850.00, "currency": "INR"},
      "net": {"value": 850.00, "currency": "INR"},
      "diagnosisSequence": [1]
    }
  ],
  "total": {"value": 850.00, "currency": "INR"},
  "contained": [
    {"resourceType": "Patient", "id": "patient-1", "birthDate": "1963-07-15", "gender": "male"}
  ],
  "extension": [
    {"url": "...session-id", "valueString": "session-uuid"},
    {"url": "...icd-version", "valueString": "ICD-10"},
    {"url": "...mapping-path", "valueString": "direct"},
    {"url": "...proposal-status", "valueString": "HOSPITAL_PROPOSED"}
  ]
}
```

```
]
}
```

Part 2: Payer Policy Gate — `services/payer_policy_gate.py`

Design Philosophy

The Payer Policy Gate is a **deterministic, payer-configurable rule engine** that answers one question before every claim submission:

"Does this claim meet our payer's standards for automatic payment, or does a human adjudicator need to review it first?"

This is the core automation value of CodePerfect for insurance companies — potentially reducing the 85% of low-complexity claims that currently require human review to requiring review for only the 15% that are genuinely ambiguous.

Trust Gate Classification

```
# Approved mapping paths: these indicate the AI found a reliable code
APPROVABLE_MAPPING_PATHS = {
    "direct",          # SNOMED → ICD crosswalk exact match (highest trust)
    "embedding",       # Vector similarity above 0.55 threshold
    "who_api_icd11",    # WHO ICD-11 API confirmed the code
    "who_api_icd10",    # WHO ICD-10 API confirmed the code
    "provider_fallback", # FHIR Condition resource fallback
    "provider_augmented", # FHIR Condition + additional pipeline enrichment
}

# These paths indicate the AI FAILED to find a code reliably – never auto-approve
REJECT_MAPPING_PATHS = {
    "no_mapping",      # SNOMED had no ICD crosswalk entry
    "embedding_failed", # Vector search found nothing above threshold
    "unknown",         # Pipeline crashed mid-run
    "no_snomed",       # SNOMED resolution completely failed
}
```

Explanation

The mapping path set classification is a clinical safety decision. `embedding_failed` means the AI searched 71,000+ ICD codes by semantic similarity and found nothing with ≥ 0.55 cosine distance — the clinical text is too ambiguous for any automated system to code reliably. Allowing such claims through would mean a human coder should have entered the code manually, and auto-approving it would legitimize a potentially incorrect or fraudulent claim without human oversight.

The Gate Runner — All 7 Checks

```
def run_payer_policy_gate(
    *,
    claim_data: dict,
    payer_policy: dict,
    org_settings: Optional[dict] = None,
) -> dict:
    """
    Evaluates the claim against the payer's policy.
    Returns a structured gate report:
    {
        gate_status:          "PASS" | "NEEDS_REVIEW",
        should_auto_approve: bool,
        reasons:              [{code, message, severity}],
        signals:              {confidence_score, risk_score, mapping_path, ...}
    }
    """
    reasons: list[dict] = []

    # Read the four key AI signals from the pipeline output
    confidence_score = float(claim_data.get("confidence_score") or 0.0)
    risk_score       = float(claim_data.get("risk_score") or 0.0)
    mapping_path     = (claim_data.get("mapping_path") or "unknown").strip()

    # Read payer-configured thresholds from the payers table
    conf_min         = float(payer_policy.get("auto_approve_confidence_min") or
0.80)
    max_risk         = float(payer_policy.get("auto_approve_max_risk") or 0.35)
    auto_enabled     = bool(payer_policy.get("auto_approve_enabled", False))
    requires_dob     = bool(payer_policy.get("auto_approve_requires_patient_dob",
True))
    requires_sex     = bool(payer_policy.get("auto_approve_requires_patient_sex",
True))
    accepted_icd_versions = payer_policy.get("accepted_icd_versions") or ["ICD-10",
"ICD-11"]
```

Check 1: Required Demographics

```
if requires_dob and not patient_dob_dt:
    _add_reason(reasons,
                code="MISSING_DOB",
                message="Auto-approve requires a documented patient date of birth (YYYY-MM-DD).",
                severity="HIGH")

if requires_sex and not patient_sex:
    _add_reason(reasons,
                code="MISSING_SEX",
                message="Auto-approve requires a documented patient sex (M/F/other).",
                severity="HIGH")

# Plausibility check: catch data extraction errors where LLM invents a future
DOB
if patient_dob_dt and patient_dob_dt.year > datetime.utcnow().year:
    _add_reason(reasons,
                code="DOB_IN_FUTURE",
                message="Patient DOB appears to be in the future; payer will require manual review.",
                severity="HIGH")
```

Check 2: ICD Version Compatibility

```
if proposal_icd_version and accepted_icd_versions:
    if proposal_icd_version not in accepted_icd_versions:
        _add_reason(reasons,
                    code="ICD_VERSION_REJECTED",
                    message=f"Proposed ICD system ({proposal_icd_version}) is not accepted for this payer.",
                    severity="HIGH")
    elif accepted_icd_versions:
        # Cannot determine which ICD version was used → always manual review
        _add_reason(reasons,
                    code="ICD_VERSION_UNKNOWN",
                    message="Could not determine ICD version for the proposal; require manual review.",
                    severity="HIGH")
```

Check 3: Mapping Quality Gate

```
if mapping_path in REJECT_MAPPING_PATHS:
    # Hard reject: AI failed to resolve the code reliably
    _add_reason(reasons,
                code="MAPPING_UNRESOLVED",
                message=f"Mapping path '{mapping_path}' is not eligible for auto-
```

```

approve.",
    severity="HIGH")
elif mapping_path not in APPROVABLE_MAPPING_PATHS:
    # Soft flag: unusual path but not disqualifying
    _add_reason(reasons,
        code="MAPPING_QUALITY_WEAK",
        message=f"Mapping path '{mapping_path}' requires review.",
        severity="MEDIUM")

```

Check 4 & 5: AI Score Thresholds with Coding Mode Adjustment

```

# Coding mode from org_settings adjusts the effective threshold.
# "aggressive" hospitals are held to HIGHER standards by their payer
# (tighter thresholds) because aggressive coding is statistically riskier.
conf_delta = risk_delta = 0.0
if coding_mode == "aggressive":
    conf_delta = 0.05    # Effective confidence threshold: conf_min + 0.05
    # (harder to pass)
    risk_delta = 0.05    # Effective risk threshold: max_risk - 0.05 (lower
    # allowed risk)
elif coding_mode == "conservative":
    conf_delta = 0.02
    risk_delta = 0.02

if confidence_score < (conf_min + conf_delta):
    _add_reason(reasons,
        code="LOW_CONFIDENCE",
        message=f"Confidence score {confidence_score:.2f} is below auto-approve
threshold.",
        severity="MEDIUM")

if risk_score > (max_risk - risk_delta):
    _add_reason(reasons,
        code="HIGH_RISK",
        message=f"Risk score {risk_score:.2f} exceeds auto-approve maximum.",
        severity="MEDIUM")

```

Check 6: Procedure Code Presence

```

# A claim without CPT codes has no line-level procedure detail for the payer.
# It cannot be converted to a valid EDI 837 Service Line (LX/SV1 segments).
cpt_codes = claim_data.get("cpt_codes") or []
if not isinstance(cpt_codes, list) or len(cpt_codes) == 0:
    _add_reason(reasons,
        code="NO_CPT_CODES",
        message="No procedure codes were proposed; payer may require manual
review.",
        severity="MEDIUM")

```

Check 7: Discrepancy Type Filter

```
# If the hospital coder flagged a discrepancy that indicates potential fraud,
# always require manual review regardless of AI scores.
discrepancy_type = claim_data.get("discrepancy_type")
if discrepancy_type and discrepancy_type in {"UNSUPPORTED_CODE", "OVERCODING"}:
    _add_reason(reasons,
                code="DISCREPANCY_FAIL",
                message=f"Discrepancy type '{discrepancy_type}' suggests a coding
mismatch; requires review.",
                severity="HIGH")
```

Check 8: Custom Payer Rules (`_evaluate_custom_rules`)

```
def _evaluate_custom_rules(*, reasons, custom_rules, claim_data, patient_dob_dt):
    """
    Four types of configurable rules (configured via PUT
    /api/v1/payers/{id}/settings):
    1. max_amount - Block auto-approve if total_billed exceeds ₹X
    2. exclude_cpt_prefix - Reject if any CPT code starts with prefix (e.g. "33"
    blocks cardiac surgery)
    3. require_min_age - Block if patient age is below N years
    4. require_max_age - Block if patient age is above N years
    """
    total_billed = float(claim_data.get("total_billed_amount") or 0.0)
    cpt_codes = claim_data.get("cpt_codes") or []

    for rule in custom_rules:
        rule_type = rule.get("rule_type", "")

        if rule_type == "max_amount":
            threshold = float(rule.get("threshold") or 0)
            if total_billed > threshold:
                _add_reason(reasons, code="CUSTOM_MAX_AMOUNT",
                            message=f"Billed ₹{total_billed:,.2f} exceeds auto-approve cap
₹{threshold:,.2f}",
                            severity="HIGH")

        elif rule_type == "exclude_cpt_prefix":
            prefix = rule.get("code_prefix", "")
            matched = [c.get("cpt_code", "") for c in cpt_codes
                       if isinstance(c, dict) and c.get("cpt_code",
            "").startswith(prefix)]
            if matched:
                _add_reason(reasons, code="CUSTOM_EXCLUDED_CPT",
                            message=f"CPT codes {matched} match blocked prefix '{prefix}'",
                            severity="HIGH")

        elif rule_type == "require_min_age":
            if patient_dob_dt:
                age = (datetime.utcnow() - patient_dob_dt).days // 365
```

```

        if age < rule["min_age"]:
            _add_reason(reasons, code="CUSTOM_AGE_TOO_LOW",
                        message=f"Patient age {age} yrs below minimum
{rule['min_age']} yrs",
                        severity="HIGH")

    elif rule_type == "require_max_age":
        if patient_dob_dt:
            age = (datetime.utcnow() - patient_dob_dt).days // 365
            if age > rule["max_age"]:
                _add_reason(reasons, code="CUSTOM_AGE_TOO_HIGH",
                            message=f"Patient age {age} yrs exceeds maximum
{rule['max_age']} yrs",
                            severity="HIGH")

```

Gate Final Decision

```

# PASS: zero reasons flagged → claim meets all policy criteria
# NEEDS_REVIEW: at least one reason → send to human adjudicator queue
gate_status      = "PASS" if not reasons else "NEEDS_REVIEW"

# Only auto-approve if: (1) payer has enabled auto-approve AND (2) gate says
PASS
should_auto_approve = auto_enabled and gate_status == "PASS"

report = {
    "gate_status":      gate_status,
    "should_auto_approve": should_auto_approve,
    "reasons":          reasons,          # Empty list = PASS
    "signals": {
        "confidence_score": confidence_score,
        "risk_score":      risk_score,
        "mapping_path":    mapping_path,
        "proposed_icd_version": proposal_icd_version,
        "patient_dob_present": bool(patient_dob_dt),
        "patient_sex_present": bool(patient_sex),
        "coding_mode":      coding_mode,
    },
}
return report

```

Real Gate Report Examples

Claim that PASSES auto-approval:

```
{
  "gate_status": "PASS",
  "should_auto_approve": true,
  "reasons": [],
  "signals": {
    "confidence_score": 0.92,
    "risk_score": 0.15,
    "mapping_path": "direct",
    "proposed_icd_version": "ICD-10",
    "patient_dob_present": true,
    "patient_sex_present": true
  }
}
```

Claim that NEEDS REVIEW (overcoding + high risk):

```
{
  "gate_status": "NEEDS_REVIEW",
  "should_auto_approve": false,
  "reasons": [
    {
      "code": "DISCREPANCY_FAIL",
      "message": "Discrepancy type 'OVERCODING' suggests a coding mismatch; requires review.",
      "severity": "HIGH"
    },
    {
      "code": "HIGH_RISK",
      "message": "Risk score 0.62 exceeds auto-approve maximum.",
      "severity": "MEDIUM"
    }
  ],
  "signals": { "confidence_score": 0.71, "risk_score": 0.62, "mapping_path": "embedding" }
}
```

Integration Flow — How FHIR and Gate Work Together

POST /api/v1/claims/submit



1. Check for duplicate session (prevent double billing)



